Natív Programozás Gyakorló

SZTE Szoftverfejlesztés Tanszék

2026. őszi félév

Technikai ismertető

- A programot C++ nyelven kell megírni.
- A megoldást a *Bíró* fogja kiértékelni.
  - A Feladat beadása felületen a Feltöltés gomb megnyomása után ki kell várni, amíg lefut a kiértékelés. **Kiértékelés közben nem szabad az oldalt frissíteni vagy a Feltöltés gombot újból megnyomni** különben feltöltési lehetőség veszik el!
- Feltöltés után a *Bíró* a programot `g++` fordítóval és a
  `-std=c++20 -Wall -Werror -static -O2 -DTEST_BIRO=1`
  paraméterezéssel fordítja és különböző tesztesetekre futtatja.
- A program működése akkor helyes, ha a tesztesetek futása nem tart tovább 5 másodpercnél és hiba nélkül (0 hibakóddal) fejeződik be, valamint a program működése a feladatkiírásnak megfelelő.
- A *Bíró* által a `riport.txt`-ben visszaadott lehetséges hibakódok:
  - Futási hiba 6: Memória- vagy időkorlát túllépés.
  - Futási hiba 8: Lebegőpontos hiba, például nullával való osztás.
  - Futási hiba 11: Memória-hozzáférési probléma, pl. tömb-túlindexelés, null pointer használat.
- A beadások számát és a beadási határidőt a bíróban lehet megtekinteni.
- A programban a `#ifndef TEST_BIRO` és a hozzá tartozó `#endif` közötti rész nem lesz figyelembe véve a kiértékelés során.
- A bíróba egyetlen `.cpp` forrásfájlt lehet feltölteni (zip fájlt nem fogja kiértékelni).
- Technikai hiba esetén a technikai hiba elhárulása után a hallgató folytathatja a ZH-t, és a ZH határidejét a hibának megfelelően módosítjuk. Ha a hallgató a technikai hiba miatt nem képes folytatni a ZH-t, akkor az adott alkalom igazolt hiányzásnak minősül.
- A ZH megírásához hivatalos fényképes igazolvány szükséges. Ennek hiányában a ZH nem teljesíthető.
- Fordítási hiba vagy figyelmeztetés esetén a beadás 0 pontot ér.
- A feladat csak terminálban oldható meg, IDE nem használható.

Általános követelmények, tudnivalók

- A programnak követnie kell az objektum-orientált programozás elveit!
- Csak a leírásban szereplő osztályokat, metódusokat és adattagokat kell megvalósítani, egyéb dolgokért nem jár plusz pont.
- Minden metódus, amelyik nem változtatja meg az objektumot, legyen konstans! Ha a paramétert nem változtatja a metódus, akkor a paraméter legyen konstans!
- string összehasonlításoknál az egyezés a pontos egyezést jelenti, azaz ha kis-nagy betűben térnek el, akkor már nem tekinthetők egyenlőnek (pl. a "piros" != "Piros")
- A leírásokban bemutatott példákban a stringek köré rakott idézőjelek nem részei az elvárt kimenetnek, azok csak a string határait jelölik. Például ha az szerepel, hogy a példa bemenetre az elvárt kimenet az, hogy `"3 alma"`, akkor az elvárt kimenet idézőjelek nélkül az `3 alma`, de a szóköz szükséges!
  - A tesztesetekben nem lesz ékezetes szöveg kiíratása.
- Az elvárt kimeneteknek karakterről karakterre olyan formátumúnak kell lennie, ami a feladatban le van írva (szóközöket és sortöréseket is beleértve).
- Ha az objektum másolása nem triviális (azaz a fordító által generált másolás nem elegendő), akkor a megfelelő másolást is meg kell valósítani.
- Minden megoldást a `solution` namespace-ben kell megírni (lásd a kiadott feladatokat)!

Kiindulási projekt, megoldás feltöltése

- **A megoldáshoz az előre kiadott osztályok módosítása szükséges lehet.**
  - Nem minden ZH esetében van kiindulási projekt.
- Feltöltéskor ezeket az osztályokat is fel kell tölteni és a módosításokat is pontozhatja a bíró!
- Egyes tesztesetekben a bíró módosított osztályt is használhat ezen kiinduló osztályok helyett, ezzel tesztelve a valóban helyes működést!

Zh alatt használható segédanyag

- **A ZH során használható segédanyag elérhető bíróban.**
  - https://biro.inf.u-szeged.hu/kozos/native/

1. Kiadott fájl

Az előre kiadott fájlba érdemes dolgozni! Itt találhatóak lokális teszt esetek is, mindenképpen érdemes ezeket megvizsgálni! Milyen paramétereket kapnak, miket adhatnak vissza, stb..

2. Feladatok

2.1. Position osztály

Az osztály egy pont x és y koordinátáját tárolja egész számként. A konstruktor ilyen sorrendben kapja a koordinátákat, alapértékük 0. Legyen mindkettőhöz konstans getter: getX, getY. A kivonás operátor két pont Manhattan-távolságát adja vissza: |x1-x2| + |y1-y2|.

2.2. Parcel osztály

Az osztály egy csomagot reprezentál. Tárolja az azonosítóját (id, string), célvárosát (city, string), célpozícióját (destination, Position), díját (fee, int) és prioritását (priority, unsigned).

Parcel konstruktor, getterek

A konstruktor a fenti sorrendben kapja az adatokat, a prioritás alapértéke 0. A díj nem lehet negatív, a prioritás 0 és 5 közé essen; különben dobjon std::invalid_argument kivételt "Invalid parcel" üzenettel. Legyen minden adathoz konstans getter: getId, getCity, getDestination, getFee, getPriority.

prefix++, postfix++, prefix--, postfix--

A ++ operátor növelje, a -- csökkentse a prioritást, amely mindig 0 és 5 között maradjon. A visszatérés kövesse a prefix és postfix operátorok szokásos szemantikáját.

+=, -=, + operátor

Egy egész szám hozzáadása növelje a csomag díját, kivonása csökkentse, de 0 alá ne mehessen. A += és -= legyen láncolható. A + új példányt adjon vissza, az eredetit ne módosítsa. Negatív összeg esetén dobjon std::invalid_argument kivételt "Negative amount" üzenettel.

==, !=, < operátor

Két csomag akkor azonos, ha az azonosítójuk megegyezik. A != az == tagadása legyen. A < szerint a nagyobb prioritású csomag kerül előre, azonos prioritásnál az azonosító lexikografikus sorrendje döntsön.

string konverzió, << stream operátor

A csomagot lehessen explicit módon stringgé alakítani és ostream-re írni. A szöveg mindkét esetben az alábbi formátumú legyen, záró sortörés nélkül. A stream operátor a kapott stream referenciájával térjen vissza.

```
Parcel[<id>; <city>; (<x>,<y>); fee=<fee>; priority=<priority>]
```

2.3. Depot osztály

Az osztály rekeszazonosítókhoz (string) csomagokat tárol. A konstruktor a központ nevét kapja, alapértéke "Kozpont". Egy rekeszben egy csomag lehet, és egy csomagazonosító legfeljebb egyszer szerepelhet a központban.

put, count, unsigned konverzió

A put egy rekeszazonosítót és egy csomagot kap, ilyen sorrendben. Ha sem a rekesz, sem a csomagazonosító nem foglalt, tároljon másolatot és térjen vissza true-val; egyébként false az eredmény. A count és az explicit unsigned konverzió a tárolt csomagok számát adja vissza.

[] indexelő operátor

A kapott rekeszazonosítóhoz tartozó csomagot adja vissza módosítható referenciaként. Konstans központon konstans referencia legyen az eredmény. Hiányzó rekesznél dobjon std::out_of_range kivételt "No locker" üzenettel.

erasePrefix, cities

Az erasePrefix a kapott csomagazonosító-kezdettel rendelkező bejegyzéseket törölje, és adja vissza a törölt mennyiséget. Az üres prefix mindenre illeszkedik. A cities a tárolt csomagok városait adja vissza setként.

~ operátor

A < szerinti első csomagot adja vissza konstans referenciaként. Üres központnál dobjon std::out_of_range kivételt "No parcel" üzenettel.

<= operátor

A kapott egész számot díjhatárként értelmezze. Az ennél nem drágább csomagokat vektorban, a < szerinti sorrendben adja vissza. Negatív határhoz üres vektor tartozik.

() függvényhívás operátor

Egy Position objektumot és egy egész távolságot kap. Adjon vissza azonos nevű új központot azokkal a csomagokkal, amelyek célpontja legfeljebb ennyi Manhattan-távolságra van a megadott ponttól. A rekeszazonosítók maradjanak meg. Negatív távolság esetén dobjon std::invalid_argument kivételt "Negative radius" üzenettel.

write, << stream operátor

A központ adatait a kapott ostream-re írja, rekeszazonosító szerint növekvő sorrendben. Minden sort sortörés zárjon. Üres központnál csak a fejléc jelenjen meg. A << legyen láncolható.

```
Depot: <name>
<locker>: <parcel>
...
```